AutoVaultLK

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing

Final Project Report

Project ID(PID)	11
Group Number	25
Mentor	Mr. Bhanuka Siriwardhana

Group Members

Registration No.	Name
230667F	Virusan T.
230670H	Vishula J.
230674A	R.P.M.Vithanage

Abstract

Buying and selling second-hand vehicles online is held back by inconsistent dealer data and by search tools that only understand fixed filters. Dealers re-type inventories listing by listing the same make or model is spelled in many ways, and a buyer who describes a need in ordinary language, such as “family friendly vehicle”, receives poor results. AutoVaultLK addresses these problems with a role-based web marketplace for buyers, dealers and administrators. Its search combines SQL filters which are structured with a rules first natural language parser, trigram matching for misspelt words, an optional language model repair step and vector-based semantic ranking. Its bulk-ingestion workflow accepts a CSV inventory file with an optional ZIP archive of images, validates and normalizes each row, resolves low-confidence rows with a language model, generates a search representation and embedding, processes the images, and records progress and rejected rows for the dealer. Imported listings are held for dealer review before publication. The platform is implemented with a React frontend, five NestJS services and a PostgreSQL database extended with pgvector and pg_trgm, and is prepared for serverless deployment on AWS. Approximately 2,000 automated unit tests pass, and load tests show that concurrent dealer uploads do not degrade buyer search at the target load of 50 buyers. The project shows that intelligent search and automated inventory processing can be added to a marketplace without losing traceability, review controls or security.

Table of Contents

Final Project Report **1**
Abstract **2**
Table of Contents **3**
1. Introduction **3**
1.1 Background of the Application Domain 4
1.2 Motivation for the Selected System Development 4
1.3 Importance and Main Purpose 4
1.4 System Overview and Outcome 4
2. Literature Review **4**
3. System Models **5**
3.1 System Requirements 5
3.2 System Design 7
3.3 Database Design 10
4. System Implementation **11**
4.1 Implementation Procedure 11
4.2 Materials 11
4.3 The Algorithm 12
4.4 Main Interfaces 14
5.1 Testing Approach 16
5.2 Unit Testing, Results and Analysis 16
5.3 Performance, Security and Failure Handling 17
REFERENCES **19**
A.1 Local Demonstration Procedure 20
A.2 Recommended Demonstration Scenario 20
A.3 Upload Contract Summary 21
A.4 Pipeline Stages 21
A.3 Validation Commands 22

1. Introduction

1.1 Background of the Application Domain

Second-hand vehicles in Sri Lanka are mainly advertised on websites such as ikman.lk [1] and Riyasewana [2] as well as on dealer showrooms and social media platforms. I notice that vehicle information is not recorded consistently across dealerships. Vehicle makes may have spellings key attributes are often given as freetext descriptions and different vehicle categories such as cars, motorcycles, vans, lorries and three wheelers require distinct specifications. Dealers with inventories often must enter listings manually which makes the process slow and prone to mistakes. Meanwhile buyers normally rely on category-based searches and filters which let buyers specify criteria but do not correctly interpret buyers' natural-language requests. The project requirements, written in the Software Requirements Specification (SRS) [3] describe a marketplace that lets buyers search for vehicles without creating an account, lets dealers upload inventory in bulk and gives administrators the power to verify listings and manage the platform.

1.2 Motivation for the Selected System Development

Two practical needs motivated the project: buyers should be able to search by intents like "low mileage hybrid SUV," and dealers should be able to upload pre-generated inventory files instead of manually entering listings. Both require a system that can standardise inconsistent data, keep original submissions for traceability and flag automated decisions having low confidence for review before published.

1.3 Importance and Main Purpose

The system aims to provide a trustworthy marketplace that improves vehicle discovery and inventory publication. Trust is supported through role separation, dealer verification, controlled listing statuses, audit logs and recorded notification delivery. Search combines exact filters for predictable constraints with semantic ranking for descriptive requests. Beyond serving as a catalogue, the platform records each dealer upload's processing history and traces every published listing to an approved workflow.

1.4 System Overview and Outcome

AutoVaultLK is a React web app, five independently deployable NestJS services, and a PostgreSQL database. The Auth and User Service is responsible for accounts, dealer verification state and tokens. The Marketplace Service owns listings, favourites, recommendations and search. The Ingestion Service ingests inventory uploads and runs the ETL workflow. The Admin Service owns approvals, reports and auditing and the Notification Service records and sends messages.The platform includes buyer, dealer and administrator interfaces, hybrid search and an ingestion pipeline designed to run locally and through AWS Step Functions.

2. Literature Review

Online Local vehicle marketplaces [1], [2] are mostly based on structured fields such as make, model, year, price and mileage. This is efficient if the values are standardised, but not so good if the buyer uses different words than are in the stored vocabulary or if the need is descriptive.

Semantic retrieval addresses the second weakness. Sentence-BERT model showed that sentences can be encoded as fixed-length vectors whose distance reflects similarity of meaning [4]. AutoVault LK uses the compact all-MiniLM-L6-v2 sentence-embedding model [5], which produces 384-dimensional vectors, and stores them in PostgreSQL through the pgvector extension [6]. An approximate nearest-neighbour index based on hierarchical navigable small world graphs [7] keeps vector comparison fast as the catalogue grows. The embedding is generated from a generated search text that unites structured values with the dealer's description, so that both contribute to the retrieval.

Semantic retrieval alone is not enough for vehicle shopping as budget, fuel type and gearbox needs to be maintained exactly. The PostgreSQL pg_trgm extension provides similarity functions and indexes based on trigrams, which are appropriate for handling spelling variation [8]. The selected design therefore combines deterministic rules, dictionary look-ups, trigram similarity and vector ranking. A language model hosted on the Groq platform [9] is consulted only when the rules cannot interpret a query or row with sufficient confidence, and every value it returns is checked against known makes, models and enumerations before use.

Bulk ingestion is a workflow problem as much as a parsing problem. AWS Step Functions provides a Map state that iterates over a collection with configurable concurrency, including large files held in Amazon S3 [10]. The project adopts a similar principle: validation, splitting, normalisation, persistence, image processing and notification are separate stages, so that the point of failure is known and a failure in one chunk does not stop the others.

When compared with the filter-based classified platforms [1] and [2], AutoVault LK adds four capabilities. It interprets natural-language queries and ranks them semantically while keeping hard constraints exact. It normalises dealer data against a curated and growing dictionary. It keeps the raw upload and reports rejected rows with proper reasons. In the end, it places automatically imported listings in a pending-review state, so that automation assists dealers without publishing those unchecked data.

3. System Models

3.1 System Requirements

Functional requirements: These are structured around three different roles. Buyers can register, verify their email address, view, filter and search vehicles, view dealer details, save favourites and get recommendations. Then dealers can register (individual or businesses), verify themselves , manage manual vehicle listings, upload CSV or JSON files (with optional vehicle images) for verified business dealers, monitor processing, download rejected-row

reports, and view and approve generated listings. Administrators can manage dealer approvals and users, view metrics and upload jobs, generate reports, curate the vehicle dictionary and audit records.

Non-functional requirements: Usability requires role specific interfaces and clear, row-level error messages. The performance objectives are an average response time of less than 500 ms for browsing and for record operations and less than 2 seconds for natural-language search when the system is warm, with the system supporting 10,000 listings, 50 concurrent buyers and 5 concurrent dealer uploads. targets are an average response time under 500 ms for browsing and record operations and under 2 s for natural-language search on warm invocations, supporting 10,000 listings, 50 concurrent buyers and 5 concurrent dealer uploads. These were tested as p95 thresholds.

Security includes password hashing, revocable refresh tokens, short-lived JWTs [11], RBAC and least-privilege database roles. Reliability uses rule-based fallbacks for language-model failures and trigram ranking when embeddings are unavailable. Stateless services and an asynchronous upload queue exist in order to support scalability, while Free Tier of AWS and Lambda limits led to the chunked, S3-referenced pipeline.

Use-case model:

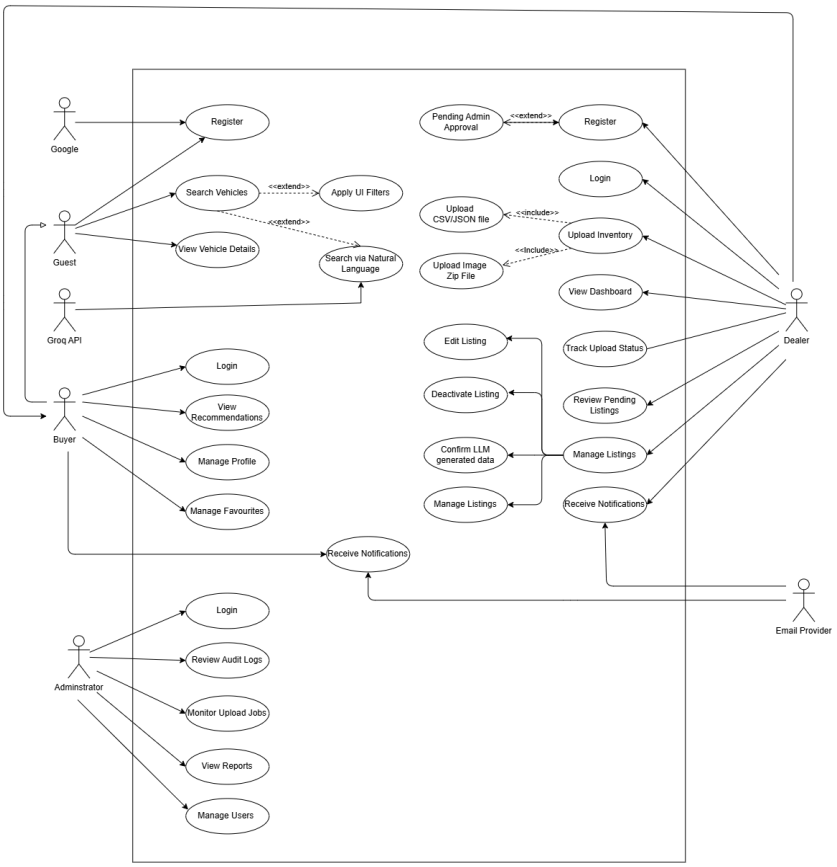


Fig. 1. Main use-case model of AutoVault LK.

Fig. 1 shows five actors. The Guest can browse, search and register, including natural-language queries through the hybrid search engine. The Buyer inherits these

capabilities and can save favourites and receive recommendations. Dealers manage listings, upload bulk inventory through the ETL pipeline, track jobs and review results. Administrators verify dealers, manage users, monitor uploads and inspect audit logs. The Notification Service sends e-mails when domain events occur.The include relationship links bulk upload to the ETL pipeline. The extend relationship links natural-language search to the hybrid search engine, which runs only when the rules cannot interpret the query with sufficient confidence.

3.2 System Design

Architecture: The system follows a service-oriented architecture (Fig. 2) described in the project's Software Architecture Document [12]. Each NestJS service keeps controllers thin, places business decisions in services and places database or external-provider access in repositories and infrastructure adapters. This follows the principle [13], in which business logic depends on interfaces such as object storage and job queue, and the environment supplies either a local or an implementation in AWS. The same ETL pipeline therefore runs locally as an in-process orchestrator and on AWS as a Step Functions state machine.

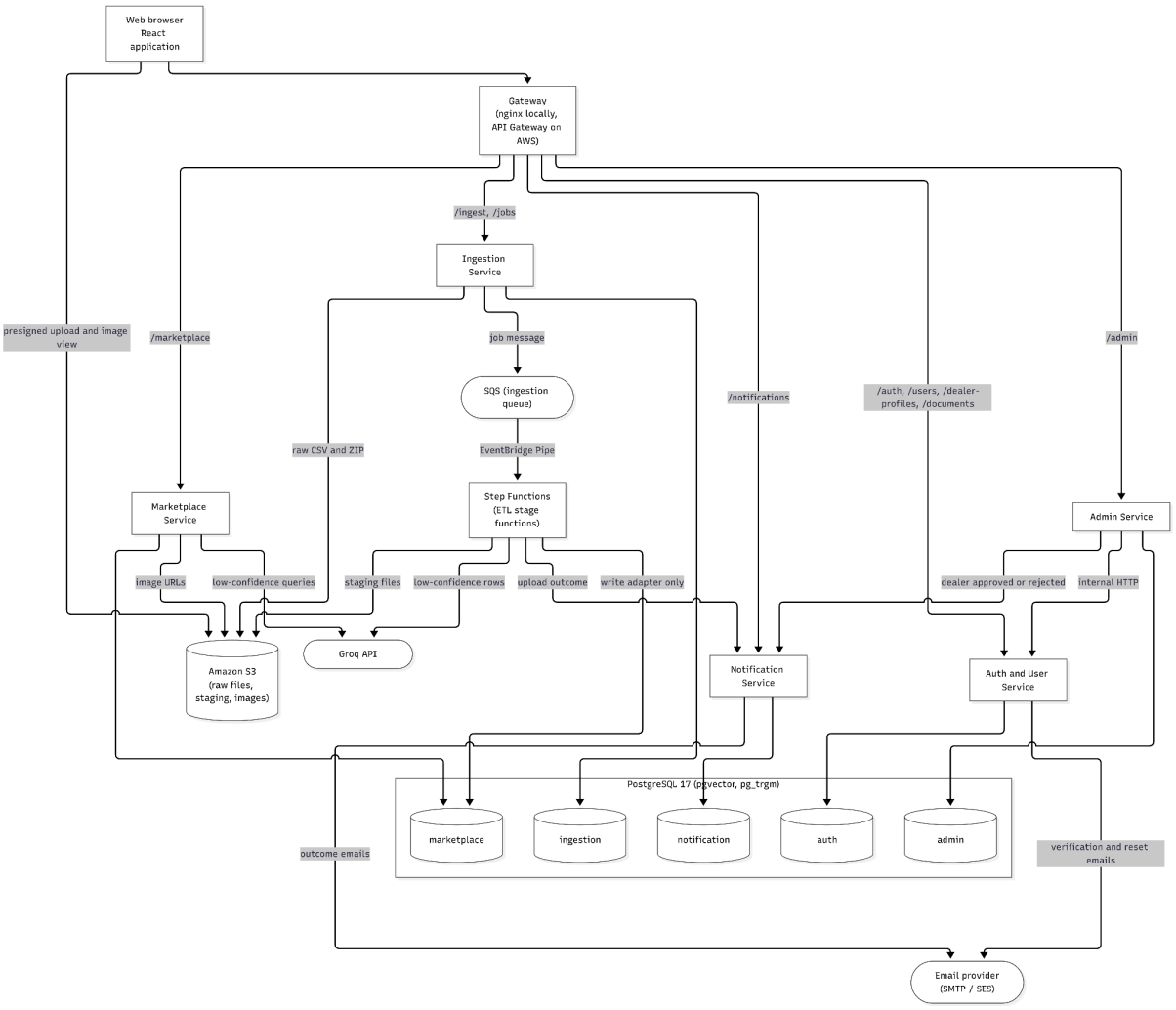


Fig. 2. Logical architecture of AutoVault LK

As Fig. 2 shows, the browser communicates with a gateway that exposes one route prefix per service. The services do share one PostgreSQL instance but are isolated based on schema, and each service holds a least-privilege database role that grants write access only to its own schema, with very narrow cross-schema read grants where unavoidable. Asynchronous work is decoupled by a queue: an upload request returns after the raw file reaches object storage and the job record is created, and a state machine then runs the ETL stages, each stage passing only a reference to its data in object storage.

Logical view: Fig. 3 presents the main classes and associations.

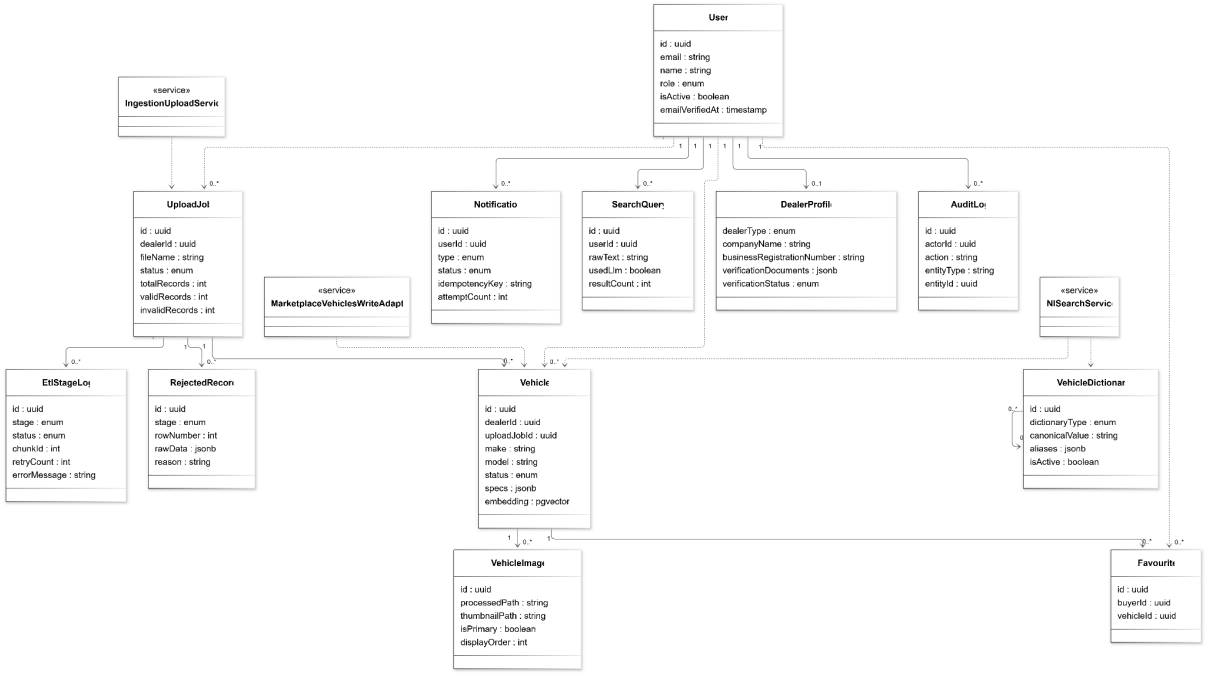


Fig. 3. Class diagram of the principal domain and service classes.

The user is the identity entity that can have roles like buyer, dealer or administrator. A dealer has one DealerProfile. Can have many vehicle listings. Vehicles have images a reference to the UploadJob that created them JSON that contains attributes specific to the type and 384-dimensional embeddings, for semantic search. UploadJob logs every ingestion run, including rejected rows and ETL stage logs. VehicleDictionary stores canonical makes/models and aliases for normalisation and search. Favourite, SearchQuery, Notification and AuditLog keep track of user preferences, searches, notifications and administrative actions respectively. At service level controllers delegate to services . Services enforce ownership , then call repositories . IngestionUploadService manages storage, jobs and queue publication, while MarketplaceVehiclesWriteAdapter exclusively writes pipeline results to the marketplace schema.

Process view: There are two main processes in the system: dealer bulk inventory upload and natural-language search.

Bulk upload: The dealer's browser uploads the CSV and optional ZIP archive directly to Amazon S3 through presigned URLs, so the files do not pass through the Ingestion Service. The browser then confirms the upload. The service makes sure the files that are supposed to be are actually there. It creates an UploadJob. It puts the work in a queue and gives back the job ID. The pipeline then checks the file. It breaks the file into pieces of 250 rows each. It processes up to 10 pieces at the time. Each row is made consistent. Only the rows that have confidence levels, less, than 0.6 are sent to the language model. Valid rows are enriched, embedded,marked and stored as pending-review listings, while rejected rows are recorded with reasons. Images are processed in parallel and matched by registration number. Finally, the job status and counts are updated and a notification is sent. Dealers poll the job status while the asynchronous processing remains isolated behind the queue.

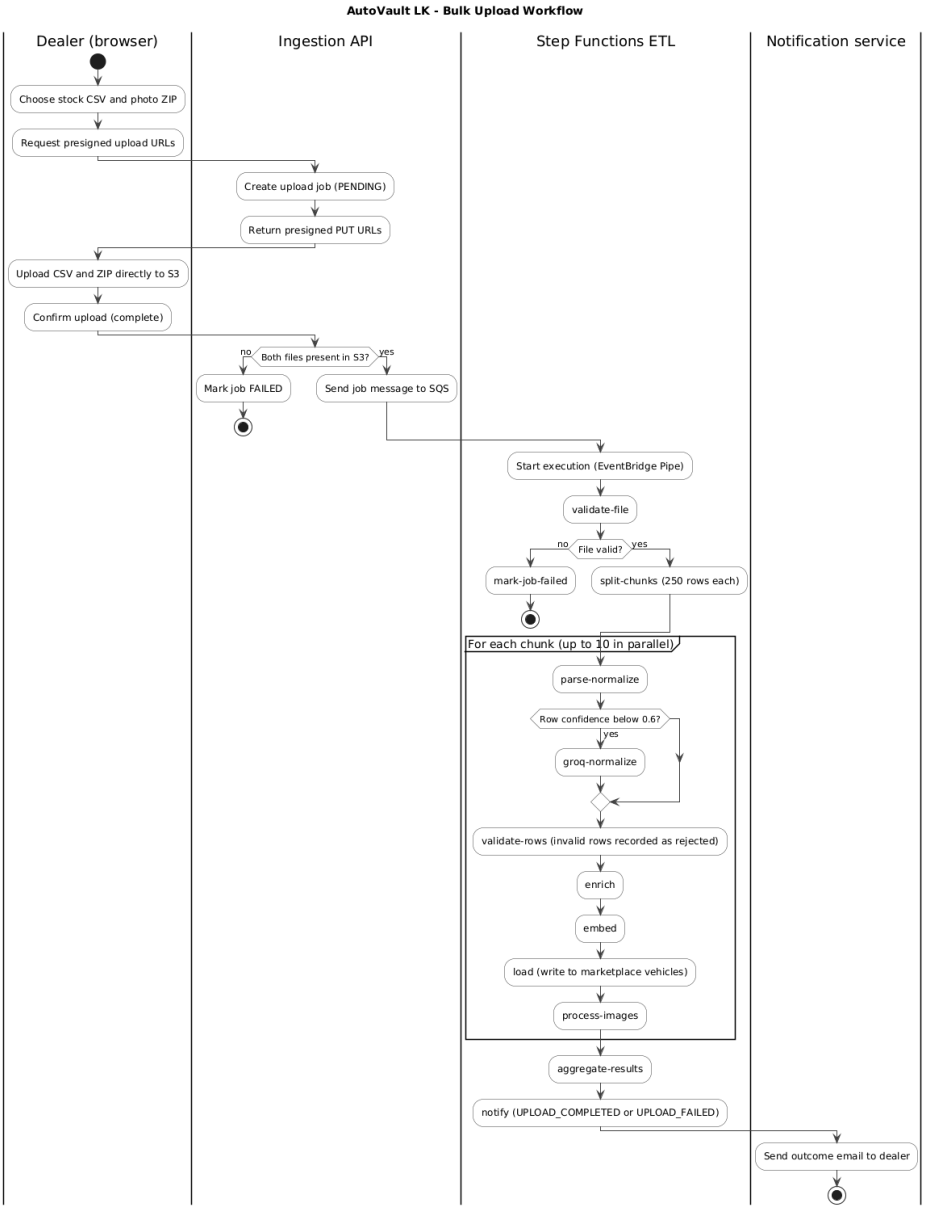


Fig. 4. ETL workflow for dealer inventory uploads.

Natural-language search: The query is first interpreted using deterministic rules. If fully resolved, filtered SQL is used directly. Otherwise, a validated language-model interpretation is obtained where needed, the remaining text is embedded, and SQL predicates are combined with similarity ranking. If either model is unavailable, the system falls back to rule-based and trigram search rather than returning an error. A visual diagram for this process has not been attached here as it will be discussed later under system implementation.

3.3 Database Design

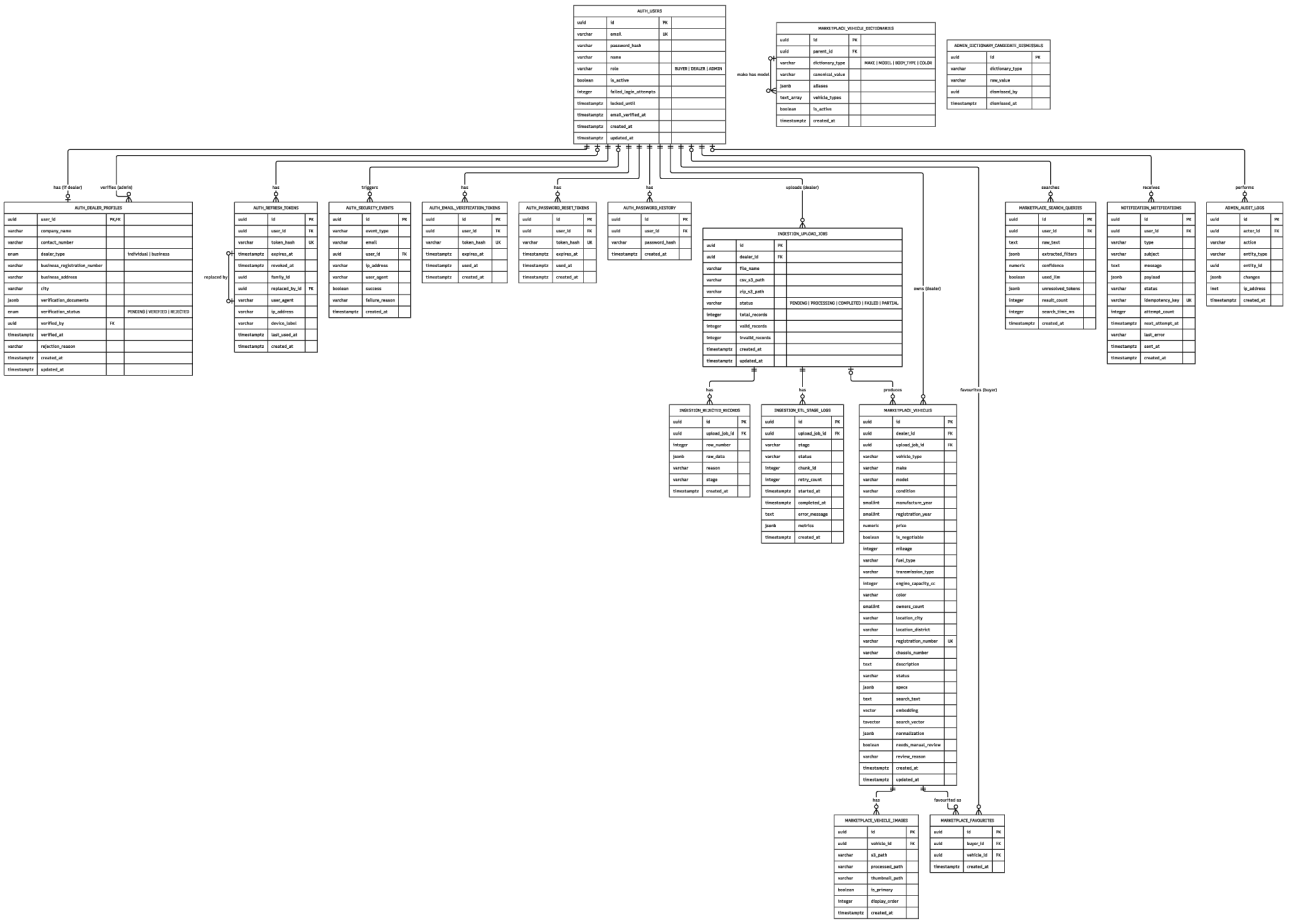


Fig. 6. Entity-relationship diagram

The database consists of a single PostgreSQL 17 instance with pgvector and pg_trgm extensions, containing 18 tables within five schemas owned by different services (Fig. 6). The auth schema is used to store users, dealer profiles, tokens, password history and security events. The marketplace schema is used to store vehicles, images, favourites, saved searches and the vehicle dictionary. The ingestion schema is used to store upload jobs, rejected records and ETL stage logs. The admin schema is used to store audit logs and dictionary-review records. The notification schema is used to store notifications. Each service has its own database role. The only controlled cross-schema writer is the ETL adapter.

Entities such as users, dealer profiles, vehicles, images, favourites and jobs are individual tables which are linked by UUID foreign keys with CHECK constraints to make sure closed vocabularies. The schema is kept denormalised in three places on purpose: type-specific attributes live in a JSONB specifications column to avoid sparse columns, and search text, text-search vectors and a 384-dimensional embedding are stored with each vehicle. Partial unique index supports only one active listing per registration number (active = draft, pending review or live). Archived and rejected listings do not block the same vehicle from being listed again, and nulls are allowed as well. And HNSW, GIN trigram and JSONB indexes support vector, text and specification search.

4. System Implementation

4.1 Implementation Procedure

TypeScript is used in the front end and in all services. The browser application is built with React [14] and Vite, and the services with NestJS [15] and TypeORM, using version controlled migrations. PostgreSQL provides transactional storage, JSONB, trigram similarity and vector operations. The MiniLM embedding model runs inside the Node.js processes through Transformers.js [16], images are resized with Sharp [17], and CSV and ZIP files are processed with streaming parsers. Docker Compose starts the database and services locally, with an nginx gateway reproducing the public gateway's routes. The AWS deployment, defined with Terraform [18], uses Lambda for the services and each ETL stage, S3 for raw files and images, SQS to decouple uploads from processing, Step Functions to orchestrate the stages, and RDS with a connection proxy. E-mail is sent over SMTP.

Four strategies mainly shaped the implementation. First, each Lambda invocation is kept small: uploads are split into chunks of 250 rows, processed at most ten at a time, with intermediate data in S3. Second, ports and adapters keep business logic independent of the environment. Third, here AI is a controlled fallback rather than a dependency: the ETL normaliser and the search engine run a deterministic parser first, consult the language model for low-confidence cases, and revert to the rule-based path on any failure of Language Model. Fourth, quality is enforced continuously through ESLint, Prettier, layered automated tests and GitHub Actions, including load tests. Seed scripts provide the vehicle dictionary, sample vehicles, dealers and an administrator account, so the full workflow can be demonstrated locally without production credentials.

The project followed an iterative, agile-style approach regarding development.A small team, with every member both implementing and testing, delivered the system in short increments over about twelve weeks. The SRS [3] and the Software Architecture Document [12] fixed the scope and main decisions at the start, and work was divided by service and feature. Each change was developed on its own branch and merged through a reviewed pull request, as the specification [3] requires. Every push and pull request ran ESLint, Prettier, type checking and the test suites through GitHub Actions.

4.2 Materials

The system does not depend on a proprietary vehicle catalogue. Its data come from four sources.

1. **A curated vehicle dictionary.** The marketplace.vehicle_dictionaries table is seeded with canonical makes, models and body types used in the Sri Lankan market, with known misspellings stored as aliases. Examples are “toyata” for Toyota and “benz” for Mercedes-Benz. The dictionary drives both normalisation and search.

2. **Dealer inventory files.** The upload contract requires 13 columns (make, model, year, price, mileage, fuel type, transmission, colour, engine capacity, number of owners, district, condition and vehicle type) and accepts about 30 optional columns such as registration number, body type and equipment. Files may be up to 25 MB, and an image archive up to 250 MB and 2,000 entries.

3. **Synthetic inventory.** As real dealer files were not available, a generator produces realistic second-hand vehicle records in clean, deliberately dirty (typographical errors, mixed case, loose formats), invalid or mixed modes, in any size, together with matching archives of synthetic images named by registration number. Test volumes ranged from 7-row fixtures to 15,000-row files and an archive of 1,800 images.

Raw uploaded files are stored unchanged, so that any processing decision can be audited or repeated.

4.3 The Algorithm

The principal algorithm is hybrid search resolution (Fig. 7). It was chosen because it must interpret free text while keeping exact constraints exact.

```
FUNCTION HybridSearch(q, vocabulary, page, sort)
  q <- ExpandBareQuery(q)      // e.g. "family friendly" -> "family friendly vehicles"

  // 1. Deterministic parsing
  R <- DeterministicParse(q, vocabulary)
       // phrases and numeric ranges (price, year, mileage)
       // exact make / model / enum matches
       // trigram fuzzy matches (see text for thresholds); style words never
fuzzy-matched
       // R.filters, R.semanticText (unresolved words), R.confidence

 // 2. Optional language-model repair
  IF R.confidence < 0.6 AND Groq is configured THEN
       G <- GroqRepair(q, R, vocabulary)
       G <- Whitelist(G, vocabulary, R)
            // drop values outside the vocabulary or permitted enumerations
            // drop style words ("sport", "luxury", ...) from resolved tokens
            // drop any model that contradicts R's make or vehicle type
       P <- Merge(R, G)                  // rules-parsed fields always win
  ELSE
       P <- R                            // also the fallback if Groq fails
  END IF


  // 3. Choose the ranking strategy
```

```
  embedding <- NULL
  IF P.semanticText is not empty THEN
       embedding <- Embed(P.semanticText)  // 384-dimension MiniLM vector, NULL on
failure
  END IF
  resolved <- P.filters contains any constraint

  IF embedding exists THEN
       cutoff <- 0.85 (1 word) | 0.78 (2 words) | 0.70 (3 or more words)
       rank   <- order by vector distance
       IF NOT resolved THEN rank.filter <- distance <= cutoff
  ELSE IF P.semanticText is not empty AND resolved THEN
       rank <- order by trigram similarity of P.semanticText
  ELSE IF NOT resolved THEN
       rank <- trigram similarity of q, keeping only matches >= 0.3
  ELSE
       rank <- requested sort (price, year, mileage, date)
  END IF
  // vector and trigram ordering apply when sort = relevance (the default);
  // an explicit price/year/mileage/date sort overrides the ordering, not the filter

  // 4. Query and graceful relaxation
  result <- SqlSearch(P.filters, rank, page)
  IF result is empty THEN
       result <- RelaxStepwise(P.filters)
       // natural-language path: descriptive word, detail filters, fuel, transmission,
       // condition, model, make, vehicle type, then price ceiling raised 20%
  END IF

  Log(q, P.filters, R.confidence, unresolved words, usedGroq)
  RETURN result
END FUNCTION
```

Fig. 7. Pseudocode of the hybrid vehicle-search algorithm.

First the algorithm applies deterministic rules. Phrases and numeric expressions (prices, years and mileage) are consumed, and known makes, models and enumerated terms are matched exactly. Remaining words are compared with the dictionary by trigram similarity. A make or model match is accepted at 0.45 or above only. Words under four characters, and styling words such as "sport", "luxury" and "family", are never fuzzy-matched, because they describe how a listing may feel rather than naming a make or model ("sport" would otherwise match "the vehicle model of Sportage").

When the confidence is below 0.6 (Confidence is the fraction of meaningful words that were resolved) , the query is sent to the language model, with a three-second default timeout and one retry. Its output is never trusted directly: every value is whitelisted against permitted enumerations, the dictionary and valid specification keys and ranges. Style words stay in the semantic text, and a model that cannot belong to the resolved vehicle type or make is discarded. Without this, "sport cars" could be corrected to an SUV-only model and combined with vehicleType = CAR, silently returning nothing. In any conflict the deterministic result wins, so the language model only fills gaps, and any failure leaves the deterministic result in place.

Unresolved text becomes the semantic text, which is embedded. If the query has structured constraints, the embedding only orders the results. If nothing was resolved, it also filters,

using a distance cut-off that depends on length (0.85 for one word, 0.78 for two, 0.70 for three or more), because shorter text embeds less precisely. Without an embedding, trigram similarity on the search text will be used. If a filtered query returns nothing, the natural-language search relaxes constraints in a fixed order. The descriptive word goes first, then the detail filters (specification filters, keywords, the registration year requirement, and the mileage and year ranges, widened as described above). Fuel type, transmission, condition, model and make are dropped next, then vehicle type. The price ceiling was raised by 20% last. A query with no hard filter left is not relaxed into the full catalogue, so a purely descriptive query returns no results. The response lists every relaxed filter. The dropdown search follows a shorter ladder that never drops make, model, fuel, transmission or price.

Each search is logged with its confidence and unresolved words. An administrator can run a promotion job over this log that adds an alias to the dictionary only when a term has recurred at least five times, scores at least 0.6 against a single canonical value and beats the runner-up by at least 0.05.

4.4 Main Interfaces

The web client provides role-specific interfaces built on a shared component library.

Public and buyer interfaces. The Home page leads into search. The Search page combines a structured filter panel (type, make and model, price, year, mileage, fuel and transmission) with a natural-language box backed by the hybrid engine, and shows results as cards. The Vehicle Detail page shows the specification, the image gallery and the dealer details, with a save action. The Saved page lists a buyer's favourites, and separate pages handle registration, e-mail verification, sign-in and password reset.

Dealer interfaces. Behind a verification gate, the Dealer Dashboard summarises inventory and upload activity. The Bulk Upload page accepts the inventory file and the optional image archive and offers instructions, a field list, a list of known makes and models, and a template download. The Upload Status and Upload History pages show each job through its stages and list rejected rows with reasons for download. The My Listings page supports row selection, bulk approval and status management, the Manual Listing page supports single entries, and the Dealer Profile page holds company details.

Administrator interfaces. The Admin Dashboard shows platform metrics. The Users page manages accounts and dealer verification, the Uploads page monitors ingestion jobs, the Reports page generates summaries, the Audit Logs page provides the append-only trail, and the Dictionary page curates makes, models and aliases.

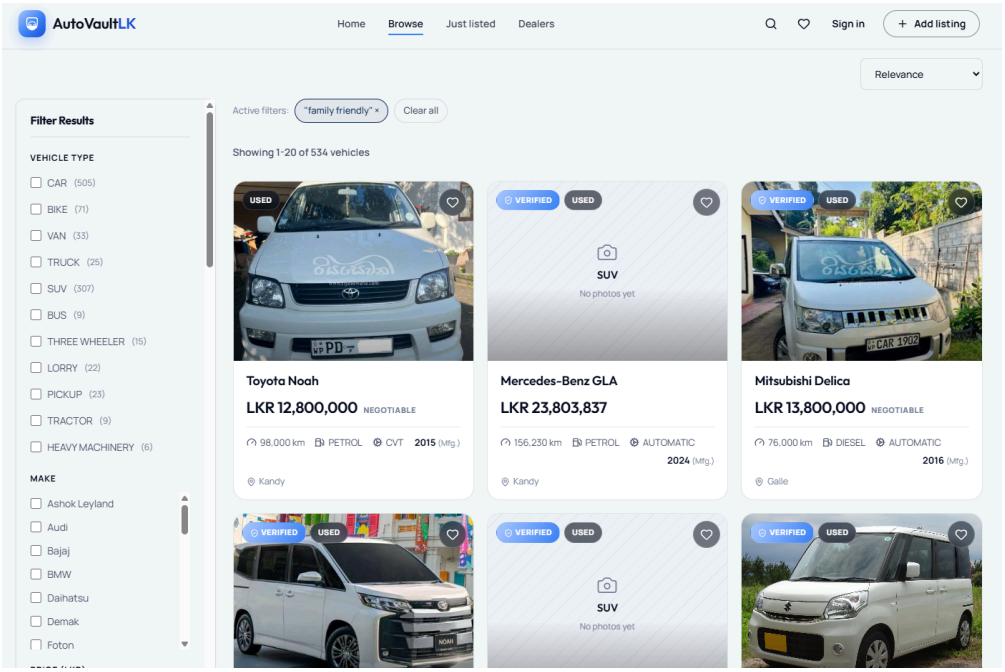


Fig. 8. Search page showing a natural-language query and ranked results.

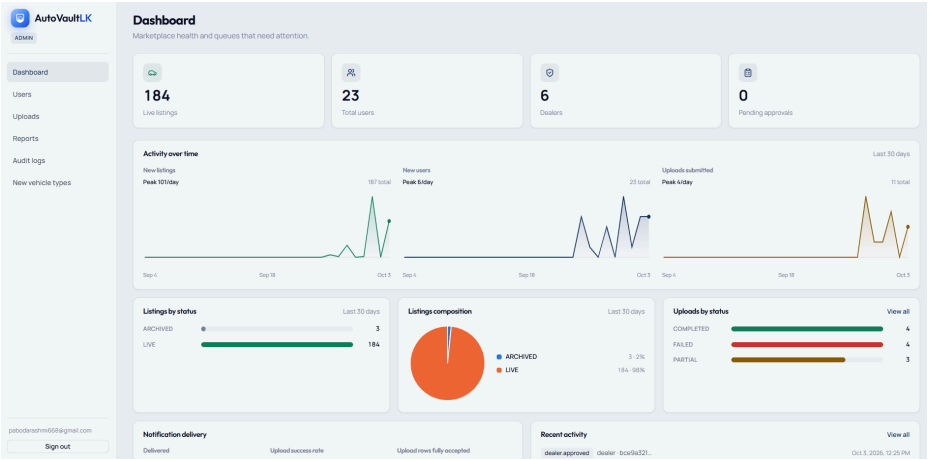


Fig. 09. Administrator dashboard.

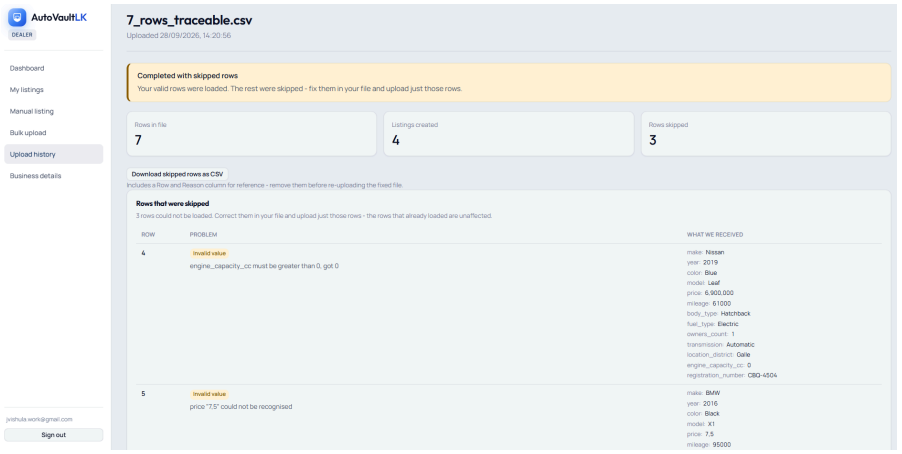


Fig. 10. Upload status page with rejected rows and corrected rows.

5. System Testing and Analysis

5.1 Testing Approach

Testing was organised across two dimensions: test level and testing technique. Unit tests verify individual classes, functions, components and pipeline stages using test doubles for external dependencies. Integration tests verify module and schema interactions against a migrated PostgreSQL database using least-privilege roles. Contract tests using OpenAPI schemas validate API responses, and end-to-end tests for dealer registration and verification, bulk upload and job status, buyer search and favourites, and administrator review and audit with Supertest and Playwright (six journey specifications).

Testing techniques included equivalence partitioning, boundary-value analysis, performance testing and failure injection. Input boundaries covered file size, entry limits, price, year, mileage and the 0.6 confidence threshold. The back end uses Jest and the front end uses Vitest with React Testing Library. Performance testing followed the Test Plan [19] using 14 k6 scripts [20].

5.2 Unit Testing, Results and Analysis

Unit suites mirror the source tree, so the test for a file can be located by its path. Table 1 summarises the automated suites executed on 2 October 2026.

Table 1. Automated test suites

Package	Test files	Tests passed
marketplace-service	55	705
ingestion-service	47	808
auth-user-service	15	86
admin-service	16	68
notification-service	10	62
web-frontend	50	385
Total	**193**	**2114**

The counts are for the default unit-level run of each package. Integration, contract and end-to-end suites are run separately against a migrated database.The largest suites are in the ingestion and marketplace services because they contain more complex behaviour, including normalisation confidence, dictionary resolution, search ranking and adaptive embedding. Tests confirmed that normalisation does not reject rows directly, row confidence is determined by the minimum supplied-field confidence, and embeddings are skipped when a query resolves completely to filters. Boundary tests also confirmed correct handling of archive and file-size limits.

Unit tests demonstrate that tested components satisfy their assertions but do not establish AWS performance, e-mail deliverability or real-world data quality; these are evaluated separately.

5.3 Performance, Security and Failure Handling

Performance. Load tests were run in two environments. Most were run on a single development laptop, with all services, the database and the load generator together, against a catalogue of about 250 live listings. The first three load levels of the end-to-end test were also run against the deployed AWS environment. Testing on AWS stopped there to stay within the free-tier allowance, because each further level adds Lambda, API Gateway and database usage that is metered and charged. The local results show that the application logic scales sensibly, and the AWS runs confirm the behaviour on real infrastructure at the lower levels. Table 2 summarises the results [21].

Table 2. Summary of performance results (local environment)

Scenario	Result
Structured search, single user	p95 53 to 95 ms (target 500 ms)
Listing detail, single user	p95 20 to 60 ms
Natural-language search with the live language model	p95 154 ms (target 2 s); one run had a 3.4 s outlier
50 buyers and 3 dealers uploading (Levels 1 and 2 of End to End Load Testing; 15 rows each, and 60 images each in Level 2)	All endpoints p95 126 ms or less, 0% errors
200 buyers and 10 dealers uploading (4 times the target)	Buyer p95 0.7 to 1.6 s with 0% errors
Buyer stress ramp to 400 users	99.99% success (1 failure in 61,050 requests); p95 599 ms
Ingestion volume, 100 to 15,000 rows	2.1 s to 129 s; about 8.6 ms per row, close to linear
600 rows with 1,800 images	244.5 s; image processing is about 16 times slower than row processing
8 simultaneous uploads against a 5-connection database pool	100% success; p95 8.76 s (queuing, not failure)

At the stated target (50 buyers with dealers uploading), concurrent ingestion did not measurably degrade buyer search. At four times the target on shared hardware, latency rose without errors; this reflects contention on one machine rather than a product limit, since each service has separate compute in AWS. Load testing on AWS covered only the first three levels. The test of the largest planned level (400 buyers and 20 dealers) was designed but not

executed, and no further AWS load testing was done because it would have exceeded the free-tier allowance. Cold-start behaviour and database connection limits at higher load therefore remain unmeasured on AWS.

Search relevance: Search was evaluated at two levels. Initially, a set of 32 labelled buyer queries (with misspelt makes and models, typos in fuel and body type, price, year and mileage phrases, and descriptive wording) were run through the rule-based parser. All 32 produced exactly the expected filters, providing 100% filter precision and recall, and the set now acts as a regression test. This measures only the query understanding. The queries were written by the team, not gathered from real buyers. Second, natural language ranking is highly dependent on listing description as suggested by a manual evaluation on ranked results. For example, “family friendly vehicle” didn’t always rank family-oriented listings higher, and adding purpose-written descriptions helped them get noticed. This supports the use of exact filters for hard constraints, and points to the need for systematic evaluation with real buyer queries.

Usability: 19 participants filled in a System Usability Scale (SUS) questionnaire, 13 participants tested the buyer role and 8 participants the dealer role (two tested both). The average SUS score was 82.1, which is above the often cited average of 68. Twelve of the 19 participants scored 80 or above. Dealers scored a bit higher (85.3) than buyers (80.6). Only one player shot less than 68. The sample is small and composed of peers.

Security: Passwords are stored using bcrypt with cost factor 12. Sessions use 15-minute JWTs, 7-day hashed refresh tokens, rotation, re-use detection. Failed sign-ins are locked for a while, registration and refresh requests are rate-limited, and e-mail verification is required. Role-based access control restricts bulk uploads to approved dealers, services use least-privilege database roles, service calls require internal keys, and audit records are append-only. An idempotency test with 15 simultaneous requests produced exactly one notification record.

Failure handling: The system is meant to degrade, not fail. Invalid files are rejected on submission and defective rows are logged without cancelling the batch. Idempotent upserts prevent duplicate listings on retries Image-processing failures don’t invalidate rows successfully loaded. If the language model fails, rule-based normalisation is used. If the embeddings fail, trigram ranking is used. Boundary testing confirmed archives over 250 MB are rejected, 245 MB archive is accepted and under database-pool exhaustion archive is queued rather than failing. Testing also showed an operational problem: unmatched photos could take up to 30 seconds per retry, causing a timeout in processing.

6. Conclusion and Future Work

AutoVault LK addresses two related issues mentioned in the introduction: it enables buyers to find relevant vehicles even if they use inconsistent language, and it allows dealers to publish inventory without unmanageable manual entry. The platform we deliver has a clear role model, an auditable administration process, a hybrid search engine and a controlled ingestion

pipeline. Dealers will deliberately hold back import of listings for review. This is important where automated normalisation can derive values from imperfect source data.

This is important because it treats data quality, search relevance and operational governance as related concerns. Buyers get both precise filtering and flexible interpretation of descriptive queries. Dealers are given job status and row level rejection reasons. Administrators still have the oversight they need to check in on dealers and review activity. The automated tests, the local environment and the created test data offer a reproducible basis for further development. During the work a few observations arose. The language model was useful especially for rows that rules could not resolve and its limitation to validated vocabulary kept it safe. The quality of listing descriptions is just as important to search relevance as the algorithm is. Ingestion time is dominated by image processing . Practical constraints for dealers with large photograph sets include archive size limits .

Main future work is Sinhala and Tamil search. Buyers in Sri Lanka often search in Sinhala, Tamil or romanised Sinhala (“Singlish”). To support them, we would need a multilingual embedding model, native-language and transliterated aliases in the vehicle dictionary and a language-detection step before parsing. In addition to that, it is expected to provide proper customer support in future**.** Support is currently limited to a contact address and telephone number in the site footer. A future release would add an in-application ticket system linked to the vehicle, listing or upload job concerned, with notifications sent through the existing Notification Service.

REFERENCES

[1] “ikman.lk: Sri Lanka's marketplace.” ikman.lk. https://ikman.lk (accessed Oct. 2, 2026).

[2] “Riyasewana.com: vehicles for sale in Sri Lanka.” Riyasewana. https://riyasewana.com (accessed Oct. 2, 2026).

[3] Group 25, “Software Requirements Specification: Second-Hand Vehicle Marketplace with Automated Inventory Processing,” University of Moratuwa, Moratuwa, Sri Lanka, 2026.

[4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. Conf. Empirical Methods in Natural Language Processing and Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, Nov. 2019, pp. 3982–3992.

[5] “sentence-transformers/all-MiniLM-L6-v2,” Hugging Face model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (accessed Oct. 2, 2026).

[6] A. Kane, “pgvector: Open-source vector similarity search for Postgres,” GitHub repository. https://github.com/pgvector/pgvector (accessed Oct. 2, 2026).

[7] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824–836, Apr. 2020.

[8] PostgreSQL Global Development Group, “pg_trgm: Support for similarity of text using trigram matching,” PostgreSQL Documentation. https://www.postgresql.org/docs/current/pgtrgm.html (accessed Oct. 2, 2026).

[9] Groq, Inc., “GroqCloud developer documentation.” https://console.groq.com/docs (accessed Oct. 2, 2026).

[10] Amazon Web Services, “Using Map state in Distributed mode for large-scale parallel workloads in Step Functions,” AWS Step Functions Developer Guide. https://docs.aws.amazon.com/step-functions/latest/dg/state-map-distributed.html (accessed Oct. 2, 2026).

[11] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015.

[12] Group 25, “Software Architecture Document, version 2.0: Second-Hand Vehicle Marketplace,” University of Moratuwa, Moratuwa, Sri Lanka, 2026.

[13] A. Cockburn, “Hexagonal architecture,” 2005. https://alistair.cockburn.us/hexagonal-architecture/ (accessed Oct. 2, 2026).

[14] “React documentation.” Meta Open Source. https://react.dev (accessed Oct. 2, 2026).

[15] “NestJS documentation.” NestJS. https://docs.nestjs.com (accessed Oct. 2, 2026).

[16] “Transformers.js documentation.” Hugging Face. https://huggingface.co/docs/transformers.js (accessed Oct. 2, 2026).

[17] L. Fuller, “sharp: High performance Node.js image processing.” https://sharp.pixelplumbing.com (accessed Oct. 2, 2026).

[18] “Terraform documentation.” HashiCorp. https://developer.hashicorp.com/terraform (accessed Oct. 2, 2026).

[19] Group 25, “Test Plan Report: Second-Hand Vehicle Marketplace,” University of Moratuwa, Moratuwa, Sri Lanka, 2026.

[20] “Grafana k6 documentation.” Grafana Labs. https://grafana.com/docs/k6/latest/ (accessed Oct. 2, 2026).

[21] Group 25, “Load and performance testing report,” University of Moratuwa, Moratuwa, Sri Lanka, 2026.

APPENDIX A. ADDITIONAL TECHNICAL DETAILS

A.1 Local Demonstration Procedure

The local environment starts PostgreSQL and the nginx gateway with Docker Compose. The five services run on the host with npm run start:dev, or all together with scripts/start-all.ps1. Database migrations and grants are applied with the database package, and the seed scripts then load the vehicle dictionary, sample vehicles, dealer accounts and an administrator account. The front end is served by the Vite development server, which proxies each route prefix to the matching service in the same way as the gateway. Services wait for database health checks before starting.

A.2 Recommended Demonstration Scenario

A complete demonstration starts when a guest searches using filters and a natural language query. Then a verified business dealer uploads a file that has validity along with an image archive. The expected result is a job. Valid rows turn into pending review listings. Rows that have errors like a misspelled make or a blank fuel type are corrected by the language model. These corrected rows are marked with their source. Rows that have errors that can't be fixed such as a year, an impossible price or a duplicate registration number show up in the rejection report, with clear reasons. The dealer approves the pending listings. Then the buyer finds those listings through search. Finally an administrator reviews the upload and Check the audit trail.

A.3 Upload Contract Summary

Item	Value
Inventory file	CSV, UTF-8 (a Windows-1252 file is converted), up to 25 MB
Image archive	ZIP, up to 250 MB, at most 2,000 entries, JPG, PNG or WebP
Required columns (13)	make, model, year, price, mileage, fuel type, transmission, colour, engine capacity, owners, district, condition, vehicle type
Photo naming	Registration number, with a numeric suffix for further photos, for example CAB-1234_2.jpg
Chunk size and concurrency	250 rows per chunk, 10 chunks at once
Language-model threshold	Rows with confidence below 0.6
Result states	Pending, processing, completed, partial, failed

A.4 Pipeline Stages

Stage	Responsibility
Validate file	Extension, encoding, header row, required columns, archive structure
Split chunks	Streams the file into chunks of 250 rows held in object storage
Parse and normalise	Dictionary, alias and trigram resolution; row confidence
Groq normalise	Language-model repair of rows below the threshold; output whitelisted
Validate rows	Single gate for all row rejections, with reasons
Enrich	Specifications, category rules, search text
Embed	384-dimension MiniLM vector
Load	Idempotent upsert as pending-review listings
Process images	Resize, thumbnail and match by registration number (parallel branch)
Aggregate	Final counts and job status
Notify	Outcome message through the Notification Service

A.5 Validation Commands

```
docker compose config --quiet
npm --prefix web-frontend test
npm --prefix web-frontend run build
npm --prefix auth-user-service run test:ci
npm --prefix marketplace-service run test:ci
npm --prefix ingestion-service run test:ci
npm --prefix admin-service run test:ci
npm --prefix notification-service run test:ci
```